

Encapsulation :- ^{blue bundling} binding data & operation together while controlling access to internal state

It is a process of eliminating non-essential details so that only the structural features of the entity remains visible to the use caller. It establishes a clear boundary ^{blue} specification and implementation.

An Interface is an area that defines the set of behaviours & implementing class must commit to. Interface cannot be instances.

A class can be implement from multiple interfaces by passing single instn.

All field are declared inside a interface

All method declared

An abstract is a class ~~is~~ marked with a abstract keyword that cannot be.

It acts as an partial blueprint intended to be extended ~~contra~~ concrete subclass.

abstract class & Interface

- It has no method body
- It must not contain $\{ \}$
- It must not contain $\& \&$
- Enclosing class contains constrain
- An Abstract method can only exist inside abstract class or an interface

Mandatory overriding

- The Non-abstract is the first concrete
- The first concrete subclass extending the abstract parent class must override & provide concrete implementation for all inherited abstract methods

Abstract methods cannot be declared as public, private, protected

abstract class A {

abstract int x, y;

}

For instance

abstract class

int x, y

}

A(int x, int y)

this.x = 30;

this.y = 30;

Store strategy pattern

Create an abstract class shape that

represent geometric shape (double area) method

(double perimeter)

void display()

Inheritance is a mechanism where in a child class derives states and behaviour from a parent class using extends keyword

It establishes has-a relationship contain the property of parent class is-a $\rightarrow$ derived from a parent class

Types

Single - class B extends class A

Multi-level -

hierarchy $\rightarrow$ class C extends class B & class A

* (Up-cast /
Down-cast)

parameterized constructor

this, name

overloading

```
class student  
{  
    y
```

```
student () {  
    y
```

```
student (String name) {  
    this.name = name;  
    y
```

Polymorphism

Greek - many form

It is the capability of a single entity (method call / Interface references) to exhibit different behaviours depending on the context or the underlying

Two types

Static polymorphism

Dynamic polymorphism

Static
Compile time
over loading
⇒ Parameters, type,
numbers.
zeroruntime

Dynamic
Run time
over riding → Binding
objects (✓) time
pointers

Compile time [method overloading]
It occurs when a single class
defines multiple method sharing
the same name but having
different parameter list.

Overloading cannot be achieved by
solely by changing the return
type of the method.

a =
b = C = a + b
S, P C " " + C)

or
Public int add (int a, int b)
{
return a + b;
}

Runtime [method overriding]
It occurs when a child class provides
a specific implementation of method that
already declared in its parent class or
interface

when calling an overridden method through a method reference Java evaluates an actual time ^{of type} on the heap at the runtime rather than the reference type

Control flow determine the order in which statement executes

Sequential execution

```
int a = 10 → set Top
int b = 10 →
int sum = a + b →
print(sum) → bottom
```

Control flow lets the program make decision and execute different blocks of code based on conditions.

if Statement

It executes the block only when condition is true

```
if (condition)
{
}
```

```
int age = 20
if (age > 0)
{
    System.out.println("Valid age");
}
```


if-else statement
provides an alternative path when
the condition is false

```
if (condition)
```

```
{
```

```
}
```

```
else {
```

```
}
```

```
int age = 18;
```

```
if (age > 18) {
```

```
    System.out.println("Eligible to  
    vote");
```

```
} else {
```

```
    System.out.println("Not Eligible  
    to vote");
```

```
}
```

if-else-if ladder

Checks multiple condition in seq
only the first true block runs.

```
if (condition1) {
```

```
}
```

```
else if (condition2) {
```

```
}
```

```
else if (condition3) {
```

```
}
```

```
else {
```

```
}
```

```
System
```

int marks = 45;

if (marks > 75) {

System.out.println("75 is
grade B");

} else if (marks > 85) {

System.out.println("85 is
grade A");

} else if (marks > 45) {

System.out.println("45 is
grade C");

} else {

System.out.println("Pass");
}

nested-if

An if inside another if

if (condition) {

if (condition) {

{

else if () {

{

else

Switch Statement

Instead of writing main if-else
we use Switch Statement

The Switch(expression) is evaluated once
The expression is compared ^{to} each
case if there is a match the statement
is matching statement is executed.

The break statement stops the switch after the match.

The default statement runs the if there is no match statement

loop

It executes a block of a code based on a condition

for

Find even or odd

while

do-while

jump

break, continue, return

Method

A Method is a reusable block of code that response - encapsulates the single block of responsibility

void display (int name, int age)
this / super.

Wrapper Class :- It is a Class whose object wraps or encapsulate primitive datatypes

byte $\xrightarrow{\text{autoboxing}}$ <Byte>

int <Integer>

Char <Character>

float <Float>

$\xleftarrow{\text{unboxing}}$

Recursion

When a method calls itself to solve problem by working on a smaller instances of that same problem

Java runtime uses a call stack to track method calls

Every call pushes a new frame (local variables/parameters & return address) onto the stack. When a method returns its frame is popped & control returns to the caller.

Static A () {}

Static B () {}

Static C () {}

Generics: It lets classes, interfaces & methods operate on a type parameter specified by the compiler giving compile time, type safety & eliminating manual casting

Collection Framework

Set of classes & interfaces that provide ready made data structure to store and manipulate group of objects

Collection is an object that can store multiple elements as a single unit.

List<patient> = new ArrayList<>();

Collection provides interfaces, [defines what a data structure can do]

Implementation - Actual classes that implements interface called Implementation.

Iterators :- Mechanism for traversing collection.

Hierarchy [structure]

Set

Alkene

Hash Set

Priority Queue

Tree Set

Malo

Hash Map

Concurrent Map

List represents an ordered collection.
Maintain Insertion order [use for DSA]
It allows duplicated & supports index based access. Elements can be accessed using index.

Array List is one of the most frequently used Java collection in DSA.

Array List ^{classname} <Integer> = new ArrayList <>()

It is backed internally by dynamic Array.
Unlike, normal Array It can grow dynamically.

Write a program to manage student marks
 Array List 85, 92, 78, 90, 65, 66

Display the marks
 Lowest Marks, Highest Marks, Average,
 Display Marks > 80
 Is 90 is there or not

Linked List implements three ^{Interfaces} instances at once which [List, queue, deque]
 which is why it shows up in so many different roles internally it is a doubly linked list
 Each node store three things
 reference to the previous node, the data,
 reference to the next node

Array List

- * It is Dynamic Array
- * Fast
- * Less overhead
- * Insertion $O(n)$ $O(1)$

Linked List

- Doubly Linked List
- Slow
- More overhead
- $O(1)$

Set Interface

A set represents a collection of unique elements. It extends the collection interface it is available in java.util package. It does not allow duplicate elements.

Page No.
 Date
 Hash Set $\langle \rangle$;

add()

contains()

remove()

Use a HashSet when you need fast membership membership and uniqueness

TreeSet

It stores unique elements in stored order

Set $\langle \text{Integer} \rangle$ name = new TreeSet $\langle \rangle$;

Queue Interface -

operation

offer() Poll() Peek()

add remove first element (without removing)

Map

It stores data as key-value pairs
key-unique and value can be any.

Map $\langle \text{Integer}, \text{String} \rangle$ name = new HashMap $\langle \rangle$;

Create a simple Student result management system where Student no. is key, Student name + marks are value.

Find the Performing adding, searching updating and removing, display the result.

Tree Map keeps its ~~tree~~ keys in its sorted order at all the times like tree set. Tree map implements a Navigable interface with the same family of neighbour finding methods and operating on keys.

map.floorKey(25) → Highest
map.ceiling(25) → Lowest
map.firstKey() → first key
map.lastKey() → Last key
map.headmap(10) → First 10 elements
map.tailmap(10) → Last 10 elements

Comparable defines a class single nature ordering implemented inside the class itself.

Comparator defines an external swapping ordering useful when you need several different ordering for the same class.

An Exception is an abnormal condition that occurs during the execution of a program and disturbs the normal flow of a ~~program~~ instruction.

Throwable is the super of all objects that can be thrown by Java exception. Every error and every exception checked or unchecked ultimately extends Throwable. It provides method getMessage(), getCause(), toString(), PrintStackTrace().

develop a simple banking application that validates user inputs using exception

Threading :-

- * Multi-threading is a process of executing multiple threads concurrently within a single program
- * Thread is the smallest unit of execution within process

UI thread, Network thread, Rendering thread

Process - VS thread

- * A Process is a running program with its own independent isolated block of memory that other process cannot directly see into Eg: Chrome
- * A thread is an execution path within a process.

Why Multi-thread

- * If everything happens sequentially the application may apply slowly. The user is stuck waiting for the download to finish completely before processing even starts and waiting processing to finish before the UI updates at all. even though these are conceptually, fairly independent activities.

- Extends
- implement

Race Condition

A race condition occurs when multiple threads access shared mutable data concurrently and the final result depends on the timing or order of their execution.

Deadlock

A deadlock occurs when two or more threads wait infinitely for resources held by each other. If a race condition is a case of threads interfering thread. A deadlock is a case of the interfering with each other progress. Neither thread does anything wrong individual but together they lock together permanently.

Synchronization

It is a mechanism to control the access the multiple thread to share the resource. It is essential in multi thread environment to provide thread interface and consistency problem. Synchronization ensures that only one thread can access the share the resource at a time.

Traffic signal simulation using Thread

Complexity Foundation

Writing a correct program answers only one program question. A second equally important question is how the efficiently program reaches that answer as the input ~~browser~~ grows. Complexity analysis is the discipline answering that second question before you even you run the code.

Eg:- Binary Search & Linear Search.

Two measures of Complexity

Time Complexity

Space Complexity.

Time Complexity is not literally how many seconds does this takes. It describes the growth rate of computational work relative to input size n .

* Asymptotic Analysis:-

Space Complexity Time records by an algorithm comes under three cases

* Best Case

* Average Case

* Worst Case

Big O notation :- This measures the performance of an algorithm by simply providing the order of growth of function. This notation provides the upper bound of a function which ensures the function never goes faster than the upper bound.

- * Omega notation:- It basically describes best case scenario which is opposite to big O notation. It is the formal way to represent lower bound for the algorithm runtime.
- * Theta (Θ) notation:- It mainly describes average case.

Space Complexity

Only fixed number of extra variables are used ~~req~~ regardless of n .

- Q * Given even number num = [2, 7, 11, 15]
 target = 9. Find the indices of two numbers that ~~apt~~ add upto the target.

Write a program to calculate $a^b \text{ mod } M$ using modular exponential. Values of a & b maybe large so that the program should ^{avoid} calculate the a^b .

```
for (i = -1; i <= 100; i++) {
    System.out.println("Hello! World!");
}
```

Time complexity : $O(n)$.
 Space complexity : $O(1)$.